Multithreaded Programming

2026 SPRING

JINHO LEE

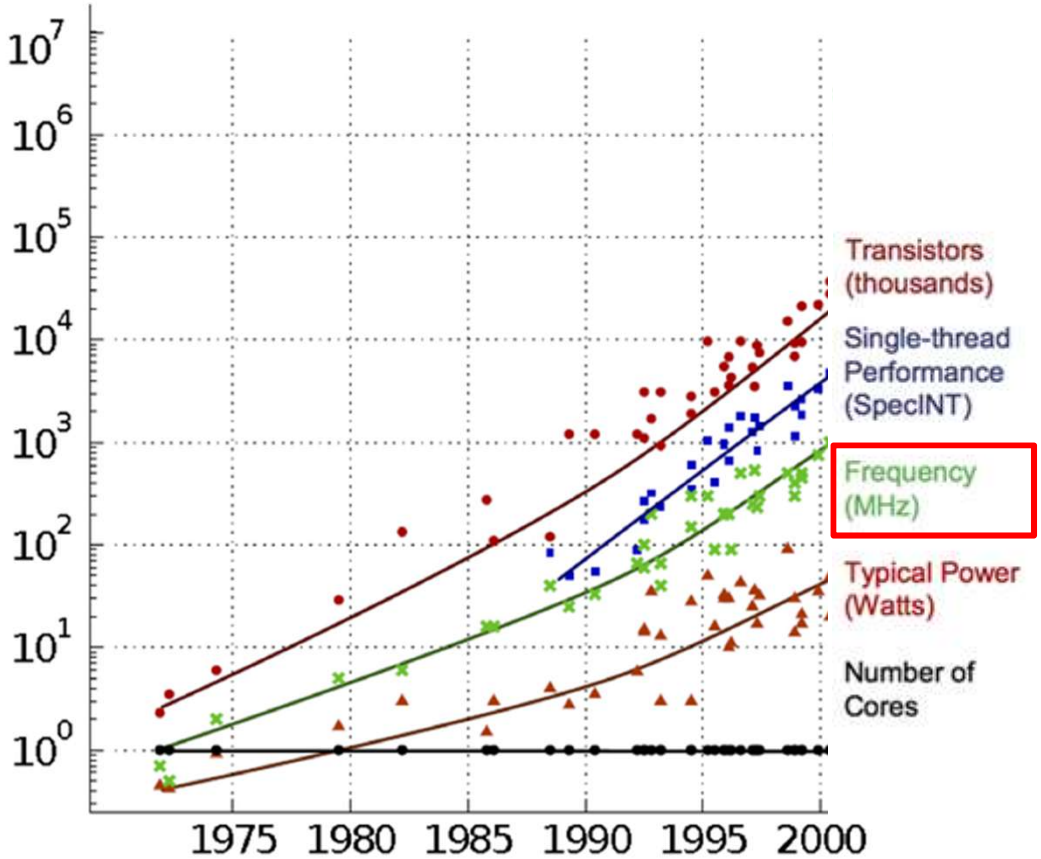
ECE, SNU

Accelerated
Intelligent
Systems Lab.

Why parallel programming?

- Can't we just live happily with single-threaded programming?
  - Like what you've done in 컴개실 프방 자구 알골… and many other courses

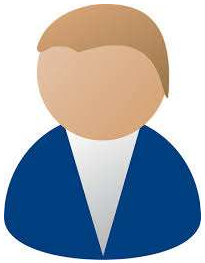
Dennard scaling era



Original data collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond and C. Batten
Dotted line extrapolations by C. Moore

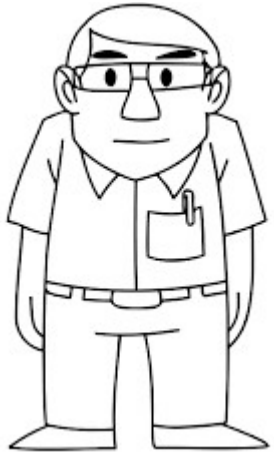
- (Roughly) the processor frequency doubled every 1.5 years
  - Thanks to Moore's law

The Magic of Dennard Scaling



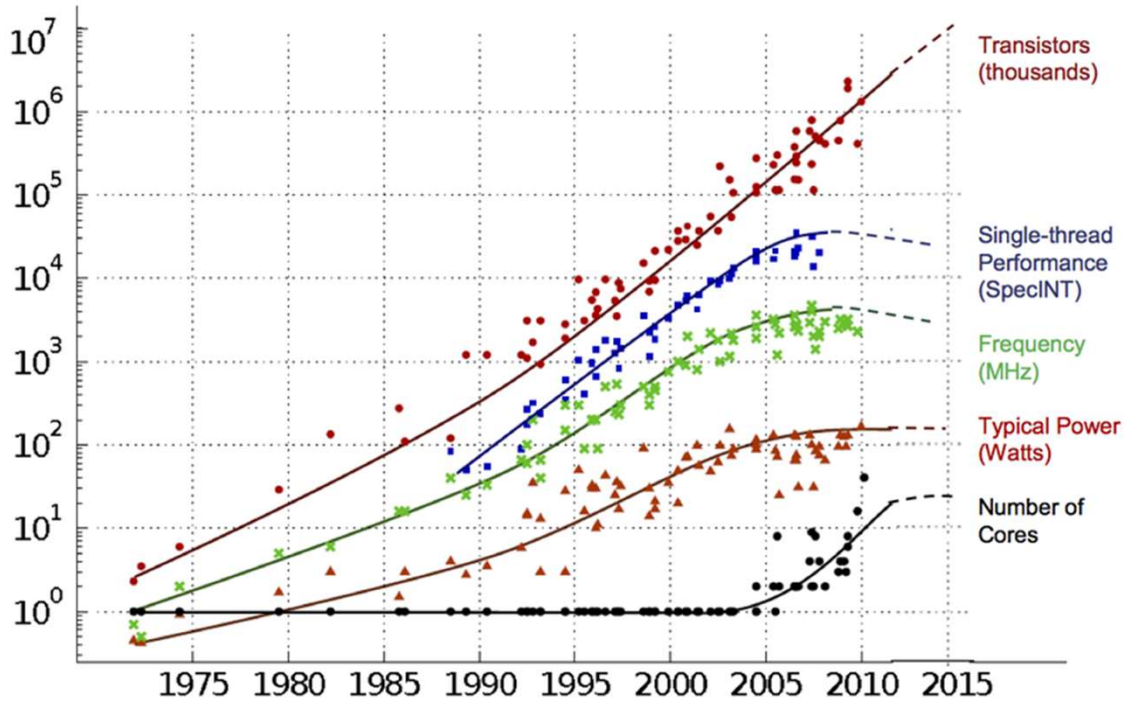
Hey boss, the game we've developed is too slow..
We only get about 10fps

Don't worry, just wait for 3 more years, and it will automatically become 40fps!



- No parallel processing, no architectural innovation needed

Dennard Scaling has Ended..



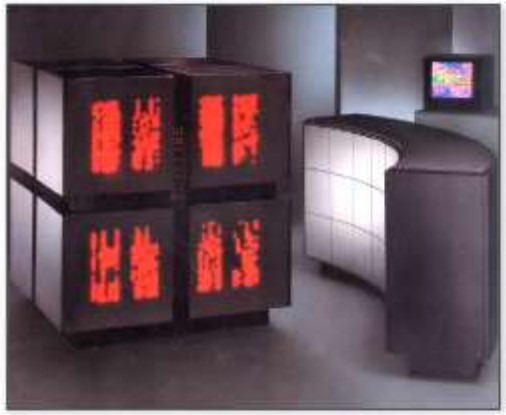
Original data collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond and C. Batten
Dotted line extrapolations by C. Moore

- Frequency growth stopped – a long story
- Moore's law is still alive – we still get more transistors
- What do we do now?
    1. More cores
    2. Different architecture (GPU, FPGA, custom IP, …)

Why parallel processing?

The answer until ~15 years ago: to realize performance improvements that exceeded what CPU performance improvements could provide

For supercomputing applications



Thinking Machines (CM2) (1987)

65,536 1-bit processors + 2,048 32 bit FP processors

SGI Origin 2000 — 128 CPUs (1996)

Photo shows ASIC Blue Mountain supercomputer at Los Alamos (48 Origin 2000's)

For database applications

Sun Enterprise 10000 (circa 1997)

64 UltraSPARC-II processors

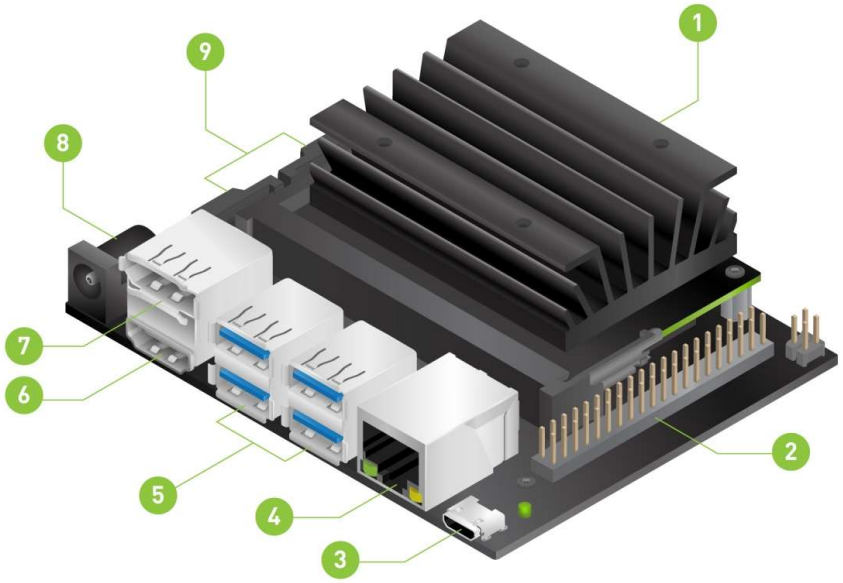
Why parallel processing?

- **The answer up until ~15 years ago**
  - To realize performance improvements that exceeded what CPU performance improvements could provide
    (specifically, in the early 2000's, what clock frequency scaling could provide)
  - Because if you just waited until next year, your code would run faster on the next generation CPU

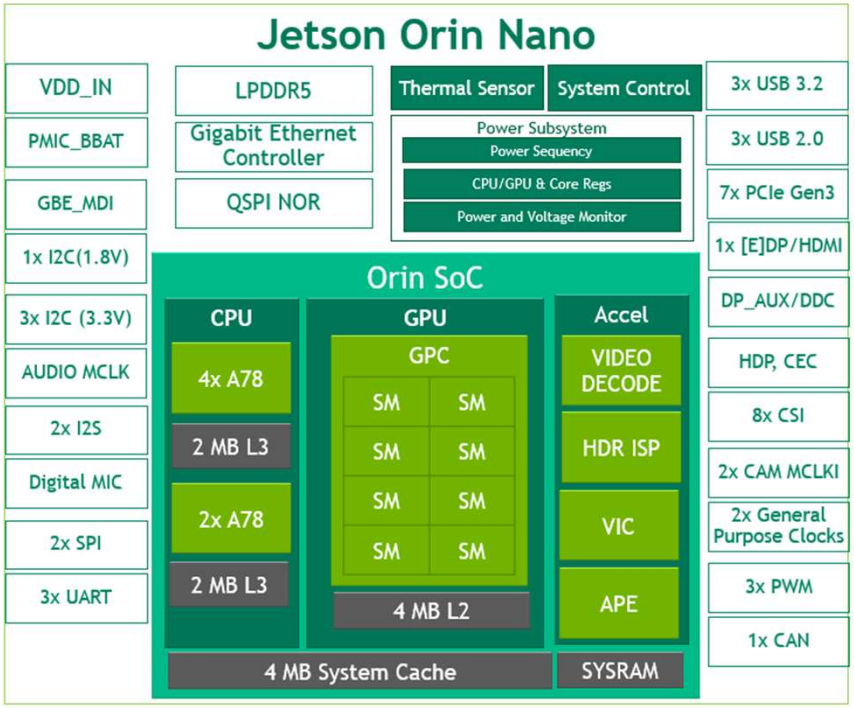
- **The answer today:**
  - Because it is <u>the primary way</u> to achieve significantly higher application performance for the foreseeable future *

Our Testboard – NVIDIA Jetson Nano

- This will be used during week 1-7



1. microSD card slot for main storage
2. 40-pin expansion header
3. Micro-USB port for 5V power input, or for Device Mode
4. Gigabit Ethernet port
5. USB 3.0 ports (x4)
6. HDMI output port
7. DisplayPort connector
8. DC Barrel jack for 5V power input
9. MIPI CSI-2 camera connectors



- 6 ARM cores, a GPU, an 8GB LPDDR5 memory
- 20 boards will be shared, not as a team, but using a job scheduler (Slurm)
  - All assignments will be conducted individually

With the Parallel Architectures..

- Now your program doesn't automatically get faster
- It's up to the programmers to utilize those parallel architectures
- Let's look at an example

Example – Sum Reduce

```
  int sum=0;
  for(int i=0;i<N;i++)  {
    sum += numbers[i];
  }

  std::cout<<sum<<std::endl;
}
```

- Classic problem, but a challenging one than you think
- Assume you have P processors (or P friends who learned addition)
- How would you split the work?

Example – sum reduce

```
sum_array[procID]=0;
for(int i=0;i<N/P;i++)  {
  int id = procId * N/P + i;
}
```

```
sum_array[procID]=0;
for(int i=0;i<N/P;i++)  {
  int id = procId * N/P + i;
}
```

```
sum_array[procID]=0;
for(int i=0;i<N/P;i++)  {
  int id = procId * N/P + i;
}
```

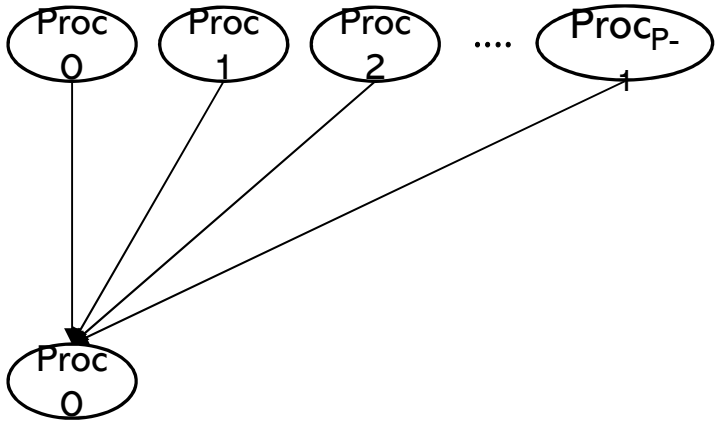
....

```
sum_array[procID]=0;
for(int i=0;i<N/P;i++)  {
  int id = procId * N/P + i;
  sum_array[procId] + = numbers[id];
}
```

- Each of the P processors got their own partial sum
- Now what?

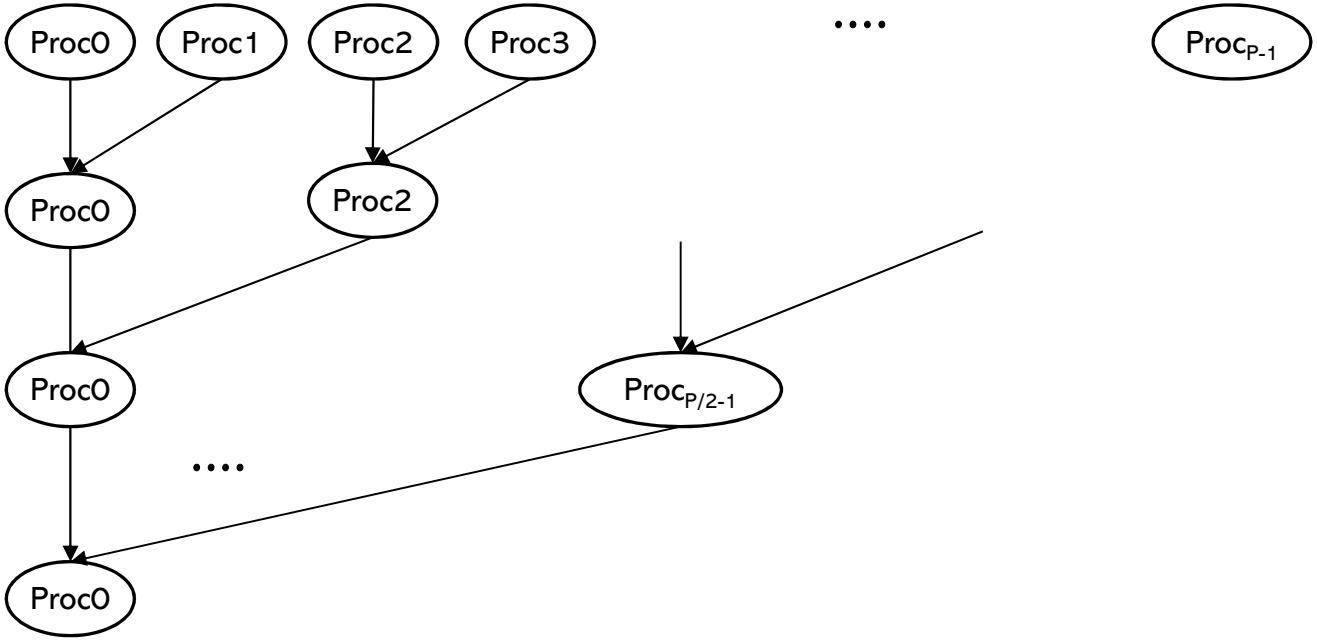
A Naïve solution

```
int global_sum = 0;
for(int i=0;i<P;i++)  {
  global_sum += sum_array[i];
}

std::cout<<global_sum<<std::endl;
```



- After the parallel execution,
  one processor becomes the master and collects all
- Is this the best?

A Better Solution



- Perform a hierarchical reduce
- Parallel additions
- Less network contention

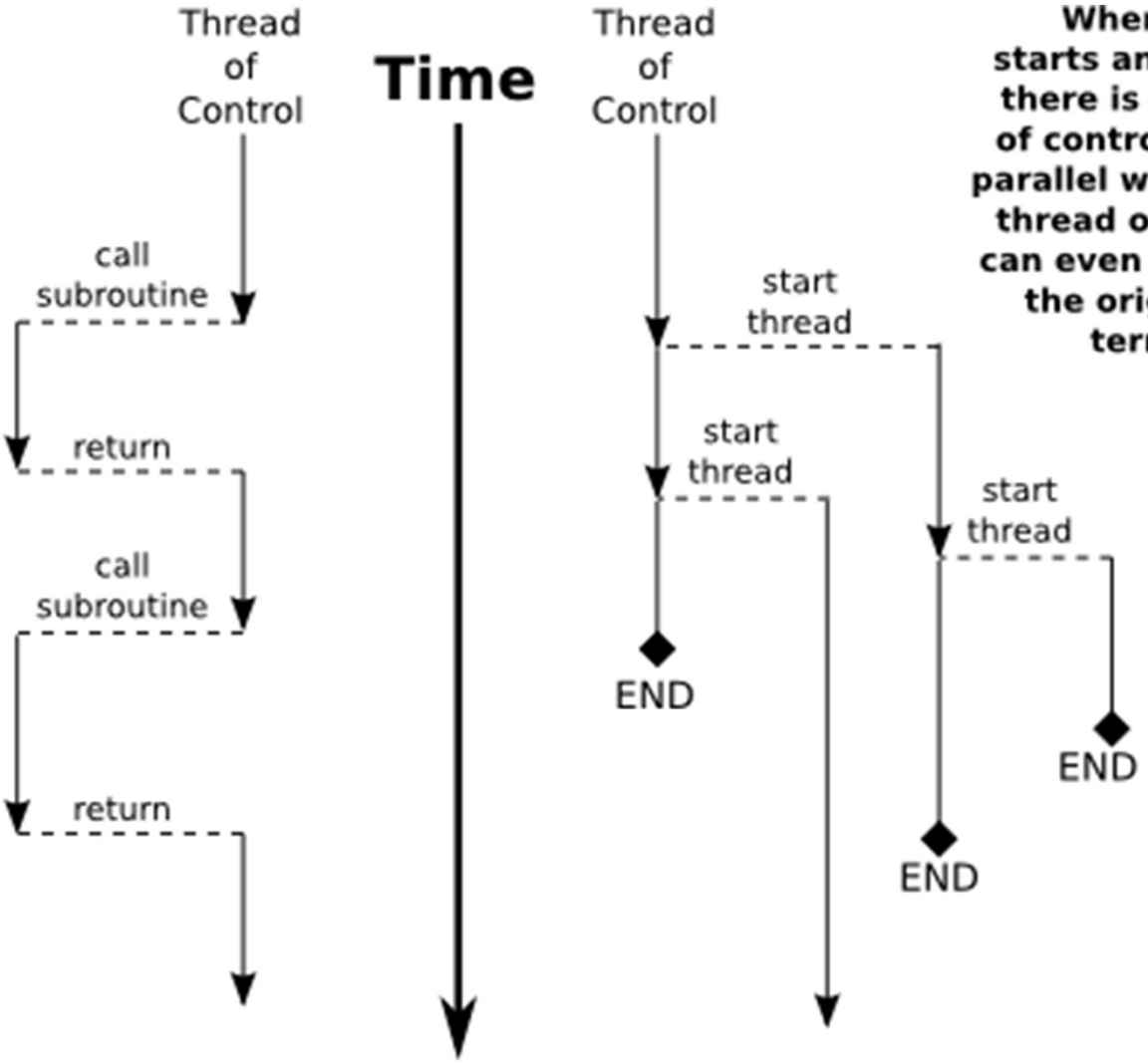
Think parallelly ! (threads)

- Humans are known to be incapable of thinking parallelly
- When you try, you end up switching between thoughts

- = threads

We use threads – ‘threads of thoughts’

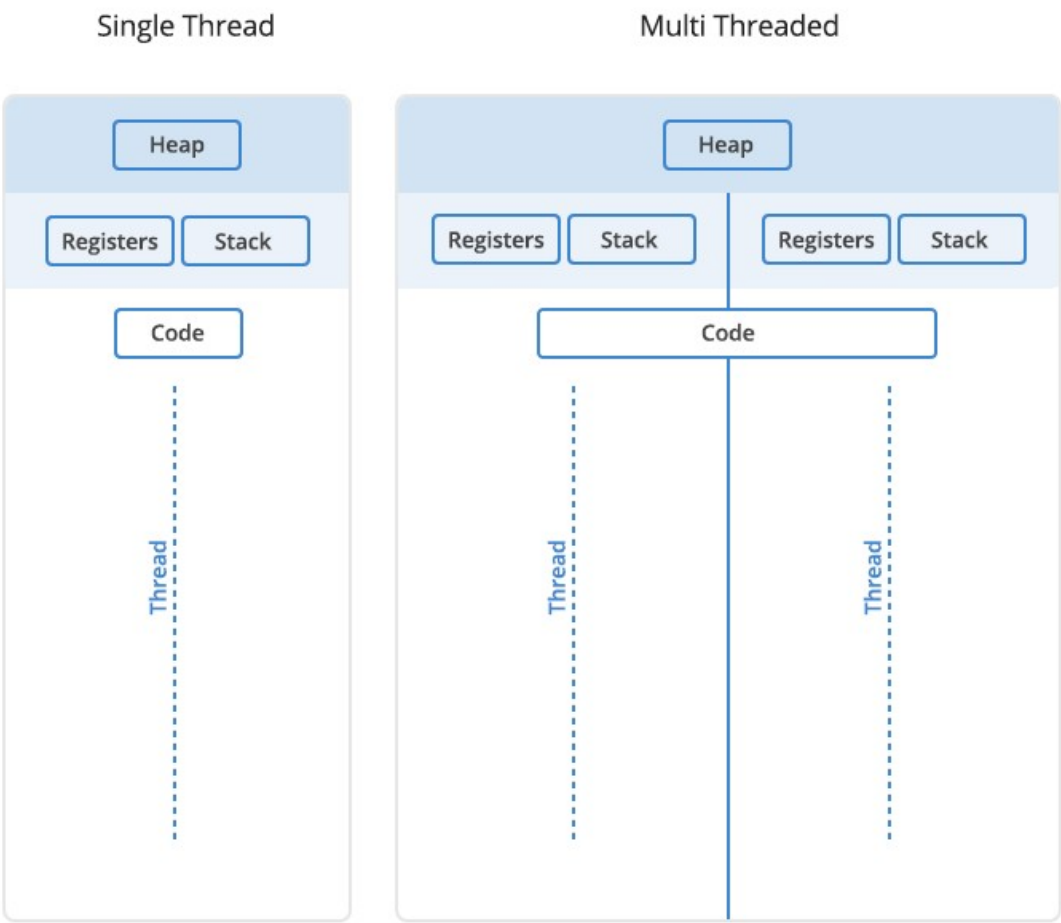
When a thread calls a subroutine, there is still only one thread of control, which is in the subroutine for a time until the subroutine returns.



When a thread starts another thread, there is a new thread of control that runs in parallel with the original thread of control, and can even continue after the original thread terminates.

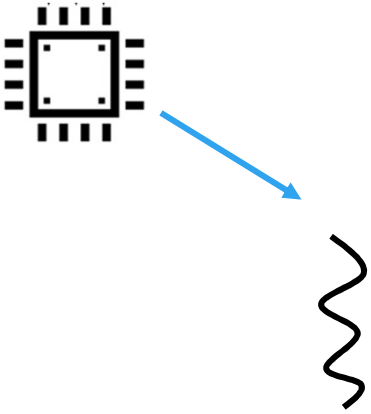
What are threads?

- Process
  - Program under execution
  - Code, data, pc, stack, register, etc
- Thread :
  - Lightweight version of a process
  - Many threads in a process share some resources
  - Shared: code, heap, address space, …
  - Private: pc, stack, registers, …



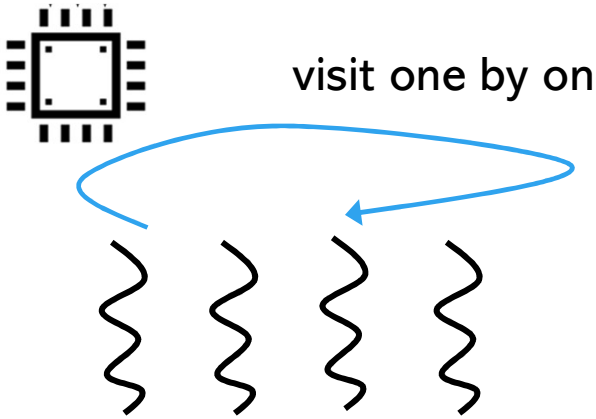
executing threads

Single core processor



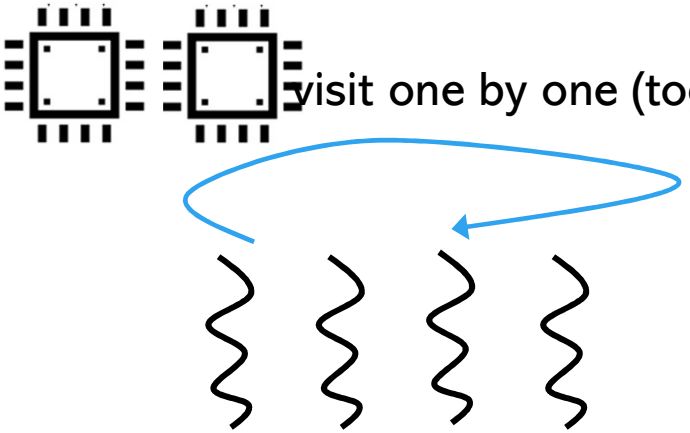
single thread

Single core processor



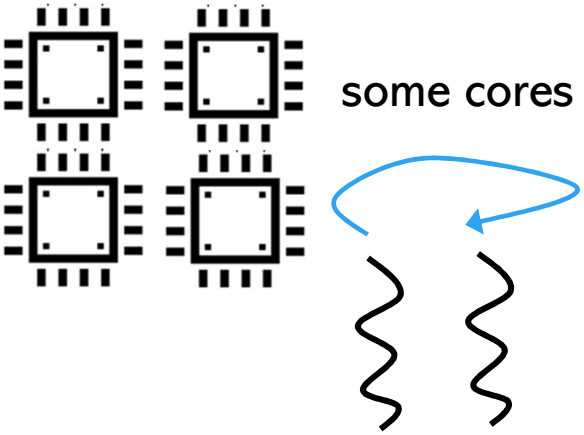
multiple threads

multi-core processor



multiple threads

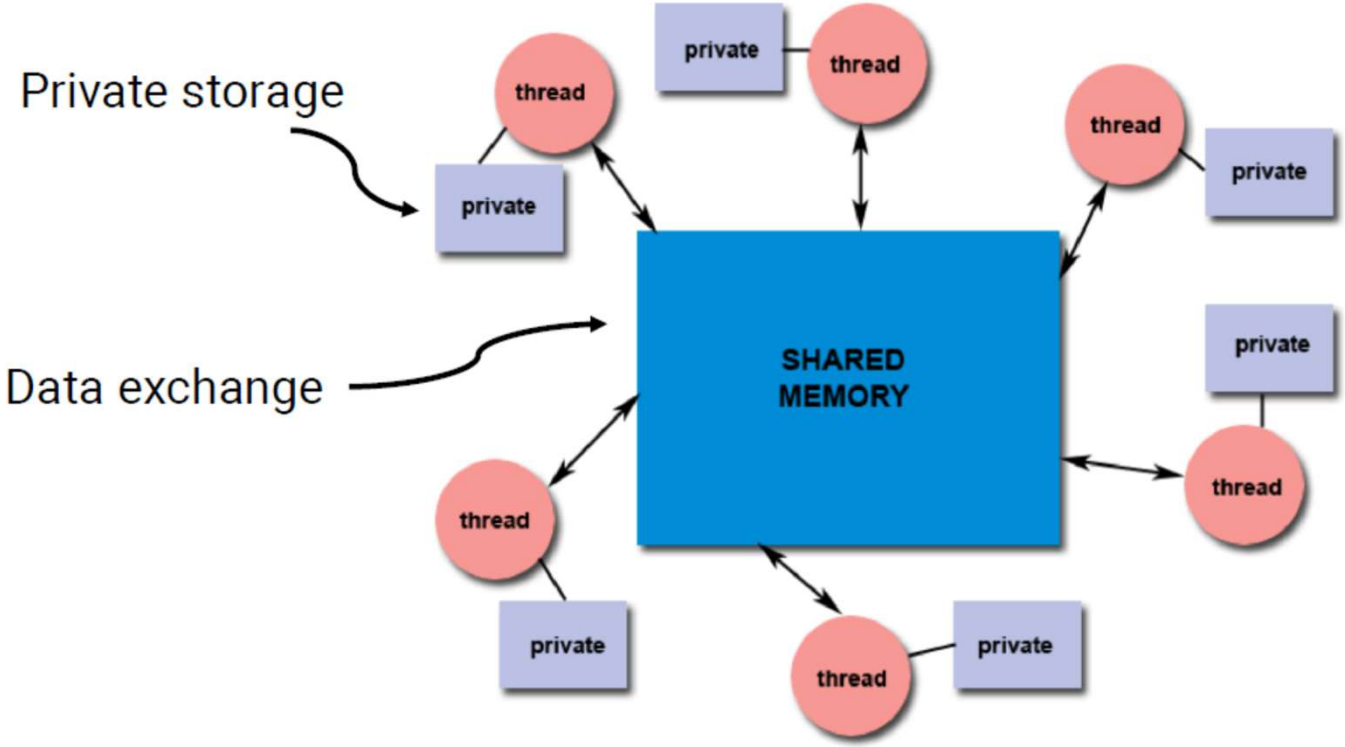
multi-core processor



multiple threads

Shared address space

- All the “threads” share the same address space
  - When they access the same address, they see the same thing
  - You might regard this as granted, but it’s not.. Think of having two desktops
- When a thread writes a value to the address space, other threads can see it
  - Which means they can work on the same array, or pointer locations



Programming with threads

- Past: we've been using a library called pthreads
- Current: C++11 now has official support for threads
  - std::thread

- Use with #include <thread>
  - Some examples you can find from web use #import <thread> but #import has been deprecated in gcc > 4.1
- Compile with
  - g++ -std=c++11 -pthread
  - Because GCC uses pthread as the backend

Example program

```
#include <iostream>
const int N = 100;
int main()
{
  int a[N], b[N], k[N], c[N];
  for(int i=0;i<N;i++) {
    ...initialize a,b,c,k
  }
  for(int i=0;i<N;i++) {
    c[i] =  k[i]*a[i];
    c[i] += k[i]*b[i];
  }
  return 0;
}
```

$$c = a \cdot k + b \cdot k$$

Creating threads

```
#include <iostream>
#include <thread>
#include <vector>
int *a, *b, *k, *c;
const int N=1024;

void mac(int tid, int num_threads)
{
  for(int i=0;i<N/num_threads;i++)
  {
    int idx = tid*(N/num_threads) + i;
    c[idx]  = k[idx] * a[idx];
    c[idx] += k[idx] * b[idx];
  }
  return;
}
int main(int argc, char* argv[])
{
...
  std::vector<std::thread> threads;
  for(int t=0;t<NT;t++) {
    threads.push_back(std::thread(mac, t, NT));
  }
  for(auto& thread: threads) {
    thread.join();
  }
  return 0;
}
```

- A program is already running in a "main thread"
- Thread object: std::thread(mac, t, NT)
  - Creating a thread object launches a new thread
- First parameter is a <u>**callable**</u>
  - There are many ways to define a callable
  - In this case, a function **mac**()
- After the callable follow the parameters to the callable

Wait for completion

```
#include <iostream>
#include <thread>
#include <vector>
int *a, *b, *k, *c;
const int N=1024;

void mac(int tid, int num_threads)
{
  for(int i=0;i<N/num_threads;i++)
  {
    int idx = tid*(N/num_threads) + i;
    c[idx]  = k[idx] * a[idx];
    c[idx] += k[idx] * b[idx];
  }
  return;
}
int main(int argc, char* argv[])
{
...
  std::vector<std::thread> threads;
  for(int t=0;t<NT;t++) {
    threads.push_back(std::thread(mac, t, NT));
  }
  for(auto& thread: threads) {
    thread.join();
  }
  return 0;
}
```

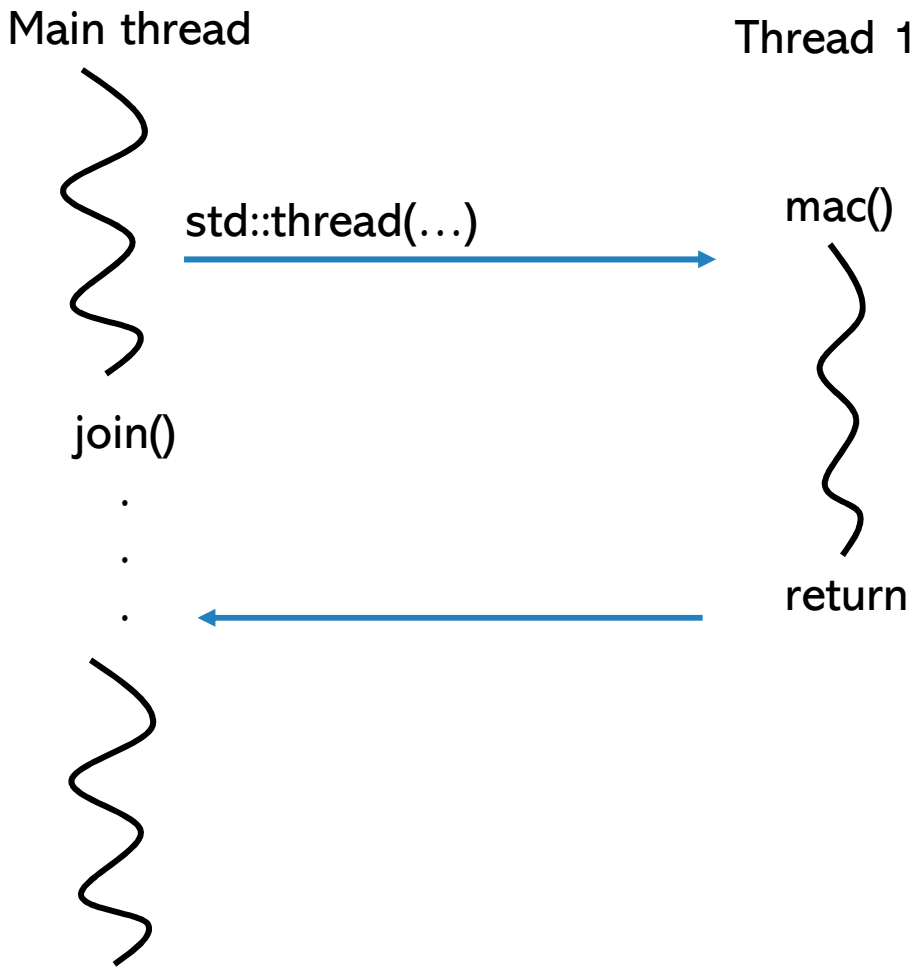
- Each thread returns when the callable returns
- With std::thread::join(), the caller thread waits until the thread object is finished
  - The thread object is stored in a std::vector in this example
  - Of course, it does not have to be a vector
- join() is used to sync threads
- Use thread.joinable() to check if it has already been joined
- Use detach() if you don't want to wait

If you don't want to wait

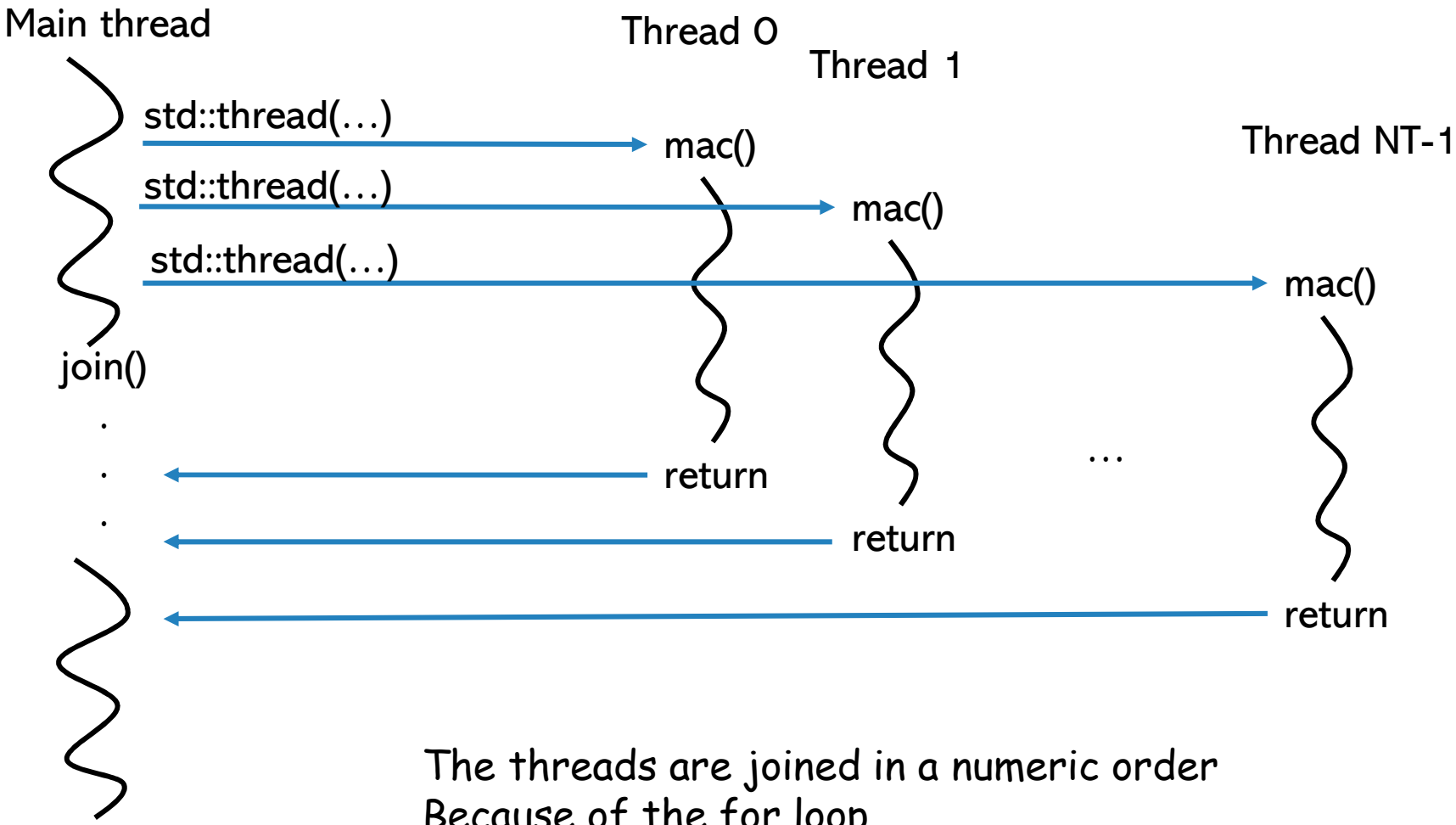
```
#include <iostream>
#include <thread>
#include <vector>
int *a, *b, *k, *c;
const int N=1024;

void mac(int tid, int num_threads)
{
  for(int i=0;i<N/num_threads;i++)
  {
    int idx = tid*(N/num_threads) + i;
    c[idx]  = k[idx] * a[idx];
    c[idx] += k[idx] * b[idx];
  }
  return;
}
int main(int argc, char* argv[])
{
...
  std::vector<std::thread> threads;
  for(int t=0;t<NT;t++) {
    threads.push_back(std::thread(mac, t, NT));
  }
  for(auto& thread: threads) {
    thread.detach();
  }
  return 0;
}
```

- Each thread returns when the callable returns
- With std::thread::join(), the caller thread waits until the thread object is finished
  - The thread object is stored in a std::vector in this example
  - Of course, it does not have to be a vector
- join() is used to sync threads
- Use thread.joinable() to check if it has already been joined
- Use detach() if you don't want to wait

Create & join

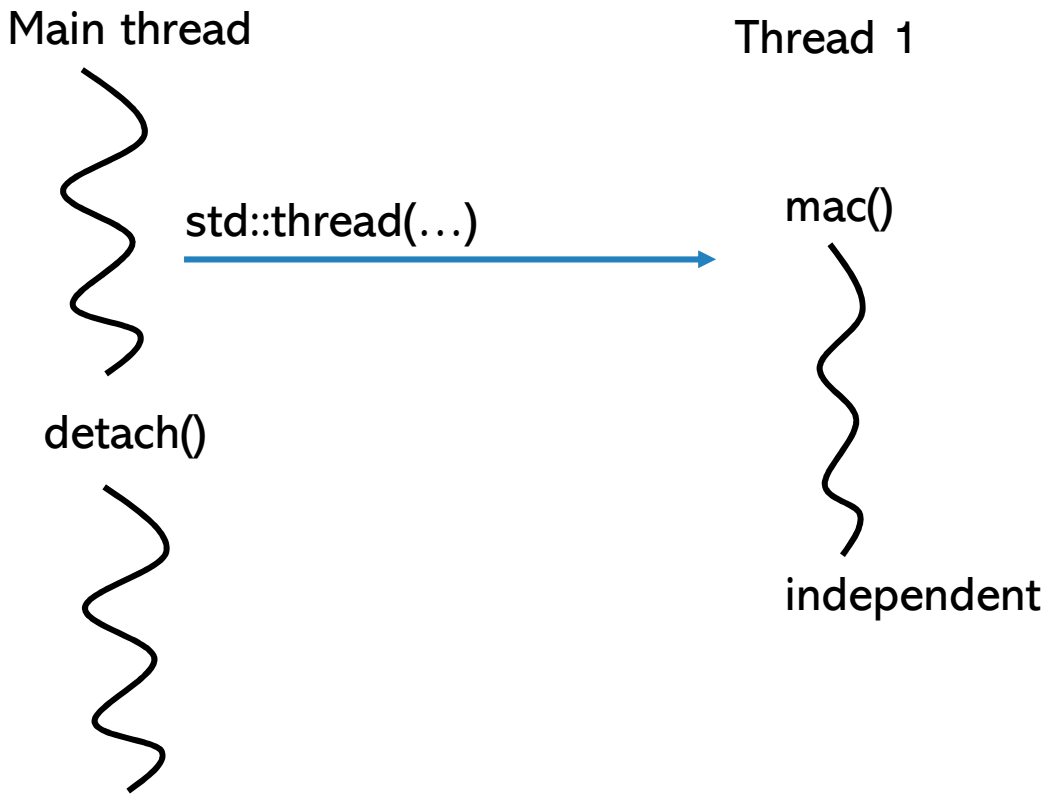


Create & join



The threads are joined in a numeric order
Because of the for loop

Create & detach



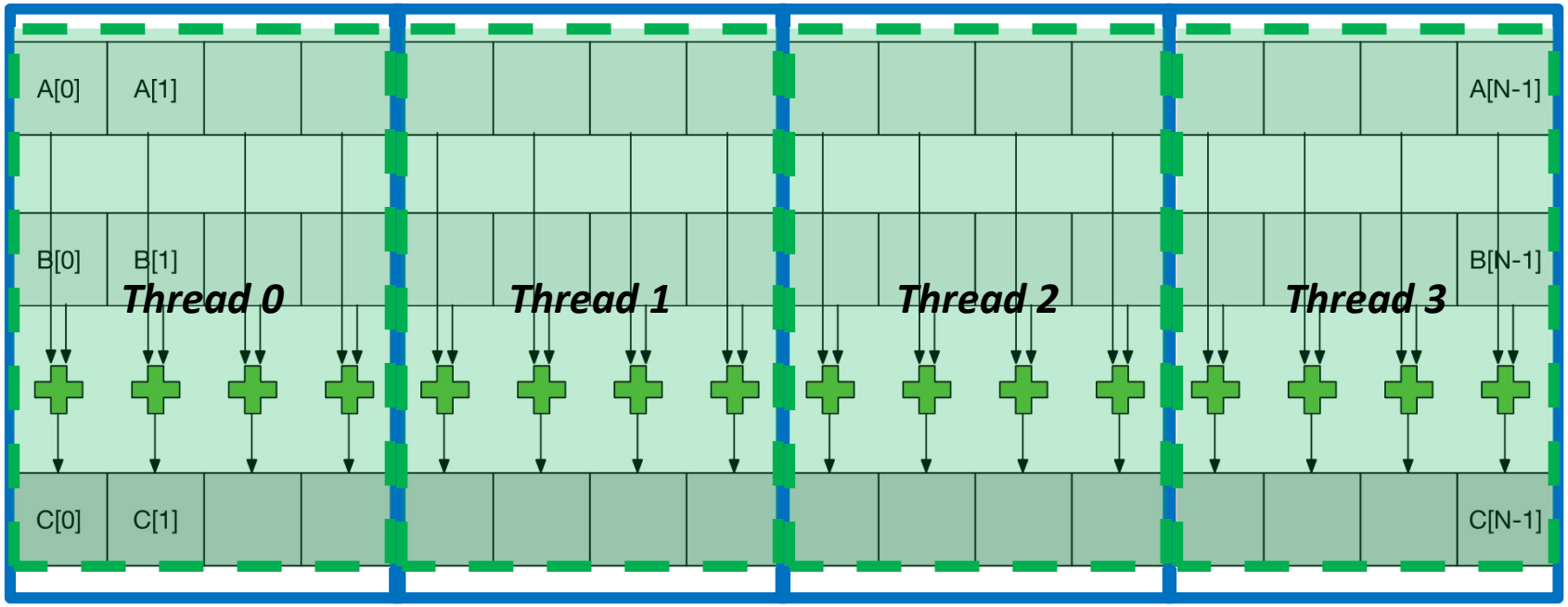
Join() or detach()

- The parent thread allocates some resources for children threads, assuming that the programmer will join them at some point
- If you are not going to join the threads, detach

- Q: what happens if I don't detach?
- A: nothing, but you will leak some resources
  (just like "what if you don't free memories after use?")

Simpler Example: Vector Addition

- Input: two large vectors (arrays) A and B
- Output: one large vector C = A+B
- Splitting the work is trivial - “Embarrassingly Parallel”

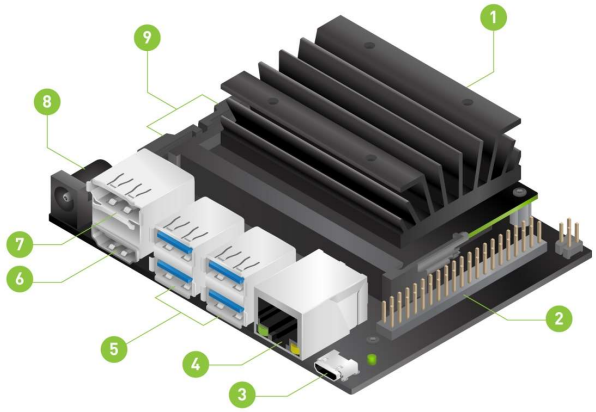


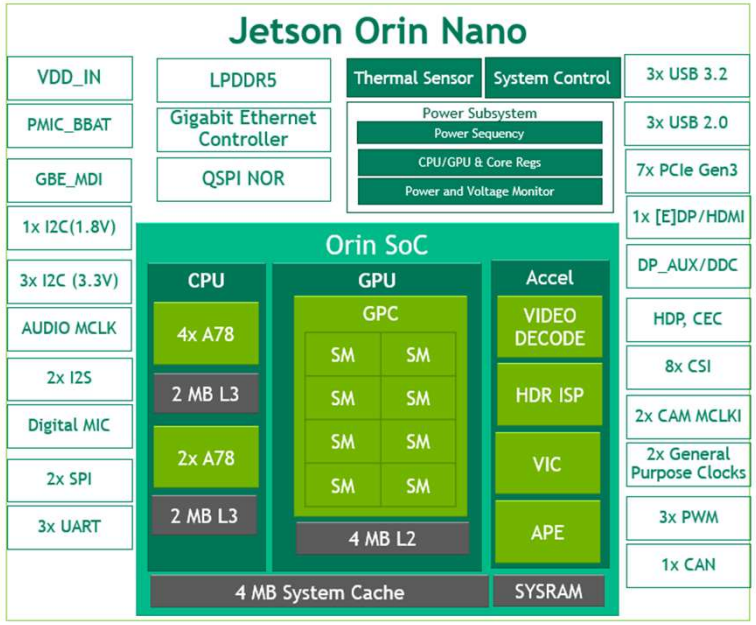
Vector Addition

```
void vadd_serial(uint32_t* A, uint32_t* B, uint32_t* C, uint32_t len)
{
    for(uint32_t i=0;i<len;i++) {
        C[i] = A[i] + B[i];
    }
}
```

- 330ms w/ len = 2^28

(On the Jetson board we are using)





Vector Addition (Parallelized)

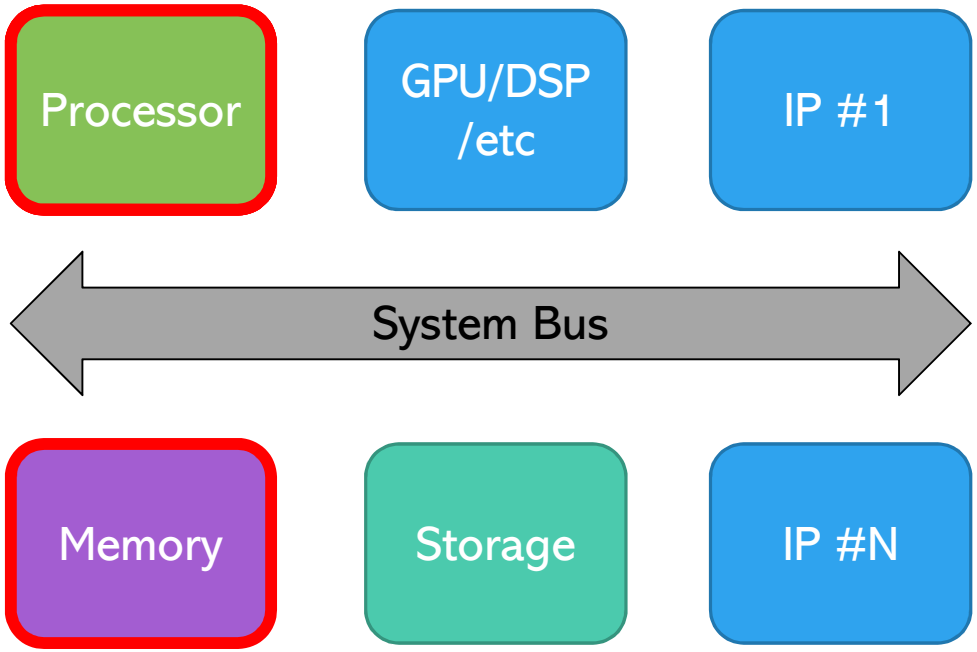
```
void vadd_worker(uint32_t* A, uint32_t* B, uint32_t* C,
                 uint32_t tid, uint32_t num_threads, uint32_t len)
{
    uint32_t worksize = len/num_threads;
    uint32_t begin = tid*worksize;
    uint32_t end = (tid+1)*worksize;
    for(uint32_t i=begin;i<end;i++) {
        C[i] = A[i] + B[i];
    }
}
int main(int argc, char* argv[])
{
    ...
    std::vector<std::thread> threads;
    for(int t=0;t<num_threads;t++) {
        threads.push_back(std::thread(vadd_worker,
                          itemsA, itemsB, results, t, num_threads, num_items));
    }
    for(auto& th:threads) {
        th.join();
    }
    ...
```

A little more care is needed for `len` being non-multiple of `num_threads`

- What execution time do you expect?
  - Single-threaded : 330ms w/ len = 2^28
  - w/ 4 threads :

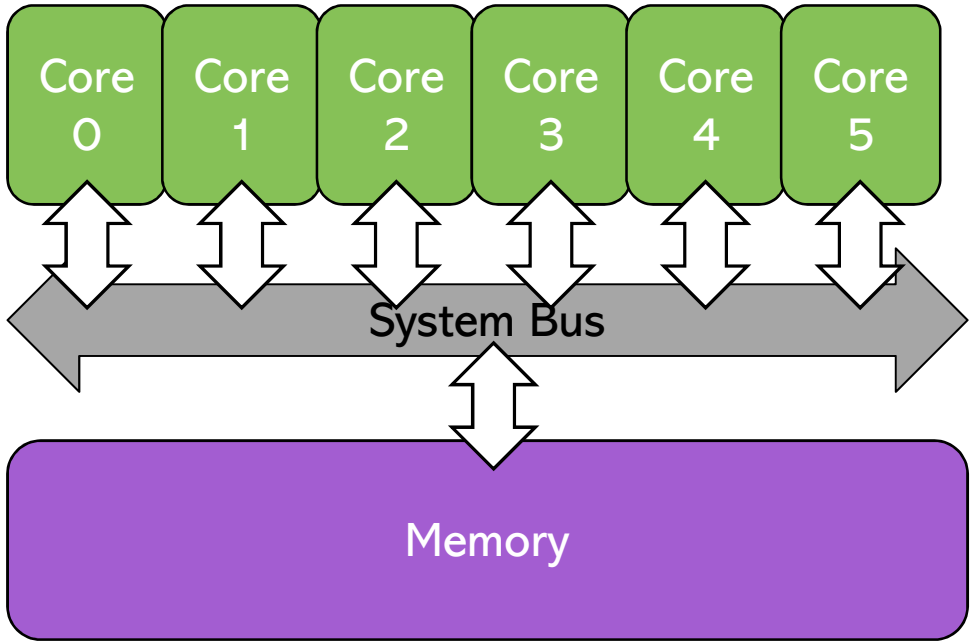
Why?

- It's "Embarrassingly Parallel" and
  it seems like we should get 4x speedup for 4 cores
- Answer: Memory bandwidth



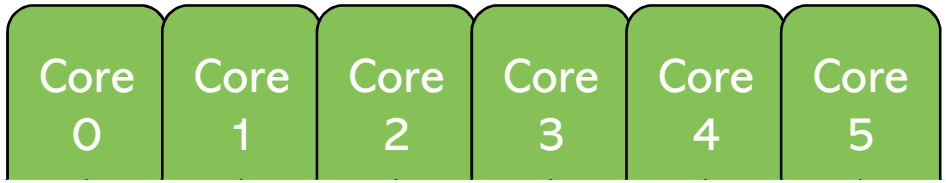
Why?

- It's "Embarrassingly Parallel" and we should get 4x speedup for 4 cores
- Answer: Memory bandwidth
- All six cores share the same bandwidth



Why?

- It's "Embarrassingly Parallel" and we should get 4x speedup for 4 cores
- Answer: Memory bandwidth
- All six cores share the same bandwidth
- We can typically get about half the nominal bandwidth from CPUs



Technical Specifications

	Jetson Orin Nano 4GB	Jetson Orin Nano 8GB
AI Performance	20 TOPS	40 TOPS
GPU	512-core NVIDIA Ampere architecture GPU with 16 tensor cores	1024-core NVIDIA Ampere architecture GPU with 32 tensor cores
Max GPU Freq.	625MHz	
CPU	6-core Arm® Cortex®-A78AE v8.2 64-bit CPU 1.5MB L2 + 4MB L3	
CPU Max Freq.	1.5GHz	
Memory	4GB 64-bit LPDDR5 34GB/s	8GB 128-bit LPDDR5 68GB/s

Why?

- It's "Embarrassingly Parallel" and we should get 4x speedup for 4 cores
- Answer: Memory bandwidth
- All six cores share the same bandwidth
- We can typically get about half the nominal bandwidth from CPUs
  - (1<<28) items * 4B/item * 3 (A/B/C) / 100ms = 32GB/s
- Remember, memory bandwidth is often the most precious resource

Technical Specifications		
	Jetson Orin Nano 4GB	Jetson Orin Nano 8GB
AI Performance	20 TOPS	40 TOPS
GPU	512-core NVIDIA Ampere architecture GPU with 16 tensor cores	1024-core NVIDIA Ampere architecture GPU with 32 tensor cores
Max GPU Freq.	625MHz	
CPU	6-core Arm® Cortex®-A78AE v8.2 64-bit CPU 1.5MB L2 + 4MB L3	
CPU Max Freq.	1.5GHz	
Memory	4GB 64-bit LPDDR5 34GB/s	8GB 128-bit LPDDR5 68GB/s

Race condition and locks

Counting program

```
int main()
{
  const int N = 1048576;
  int *array= new int[N];
  for(int i=0;i<N;i++) {
    array[i] = 0;
  }
  int count=0;
  for(int i=0;i<N;i++) {
    if(array[i] == 0) {
      count++;
    }
  }
  std::cout<<"there are "<<count<<" zeros"<<std::endl;
}
```

- Make it multi-threaded?

Counting program

```
#include <iostream>
#include <thread>
#include <vector>

void worker(int* input, int start, int size,
int* output)
{
  for(int i=0;i<size;i++) {
    if(input[start+i]==0) {
      (*output)++;
    }
  }
}
```

```
int main()
{
  const int N = 1048576;
  const int NT = 4;
  int *array= new int[N];

  for(int i=0;i<N;i++) {
    array[i] = 0;
  }

  int count=0;
  std::vector<std::thread> threads;
  for(int t=0;t<NT;t++) {
    //assume N is a multiple of NT
    int size=N/NT;
    int start = t*size;
    threads.push_back(std::thread(worker,
array, start, size, &count));
  }
  for(std::thread& thread:threads) {
    thread.join();
  }

  std::cout<<"there are "<<count<<"
zeros"<<std::endl;
}
```

- Happy?

```
there are 502841 zeros
```

less then 1048576
(different every time)

Under the scene..

Thread 0	Count	Thread 1
	0	
Read count (0)		
		Read count(0)
count++ → 1		
		count++ → 1
Write count	1	
	1	Write count

- You will practically never get the full count

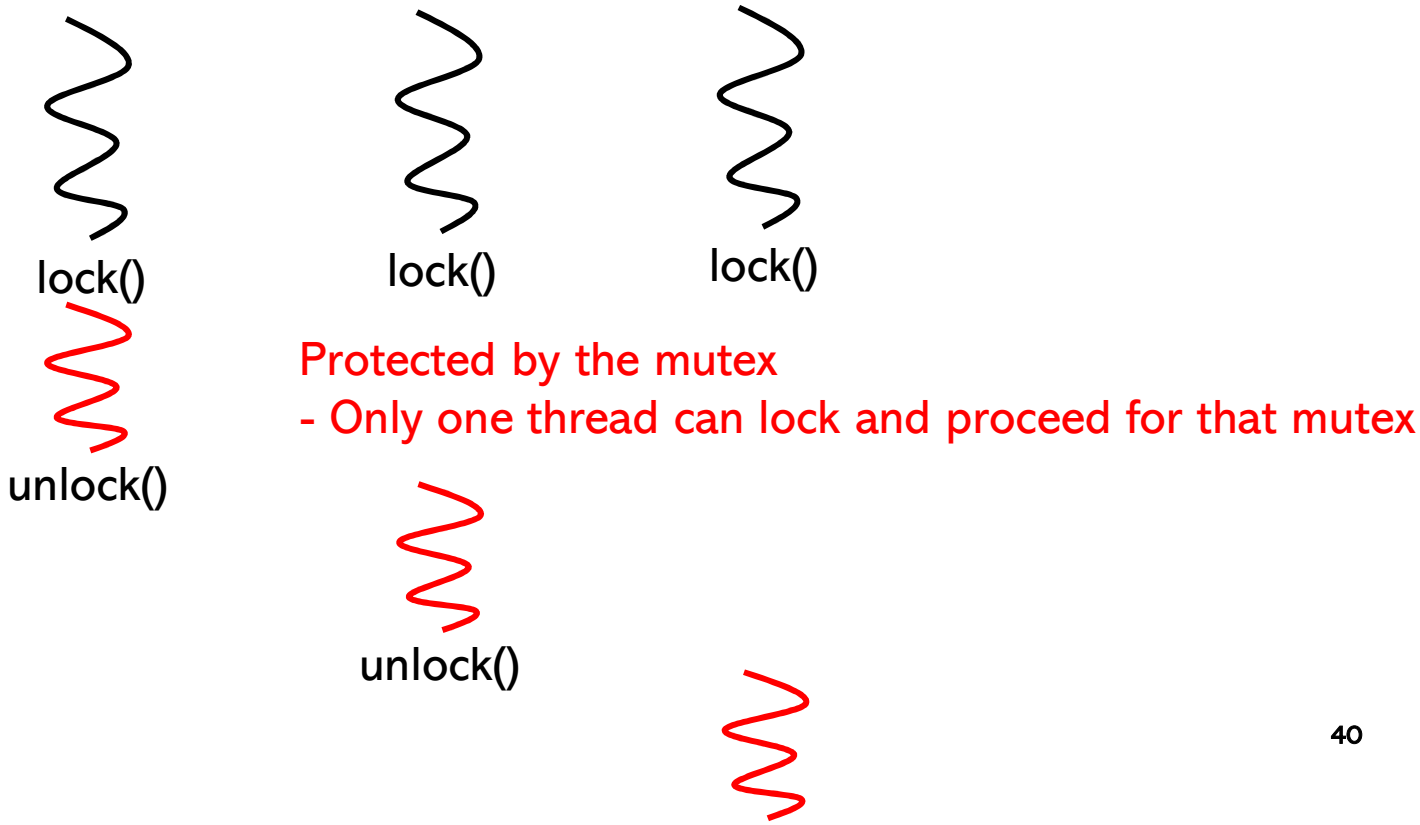
Race condition

- Problem: the read – add – write operations take time, and can be interrupted
- Race condition: the result depends on the order of the executions

- It's the programmer's responsibility to handle this

Mutex

- Mutex: a special variable
  - Locked / unlocked
  - Only one thread can lock the mutex
  - Others attempting to lock the mutex have to wait until the mutex is unlocked
  - Refer to OS / ComArch course for how it is implemented



Using mutex

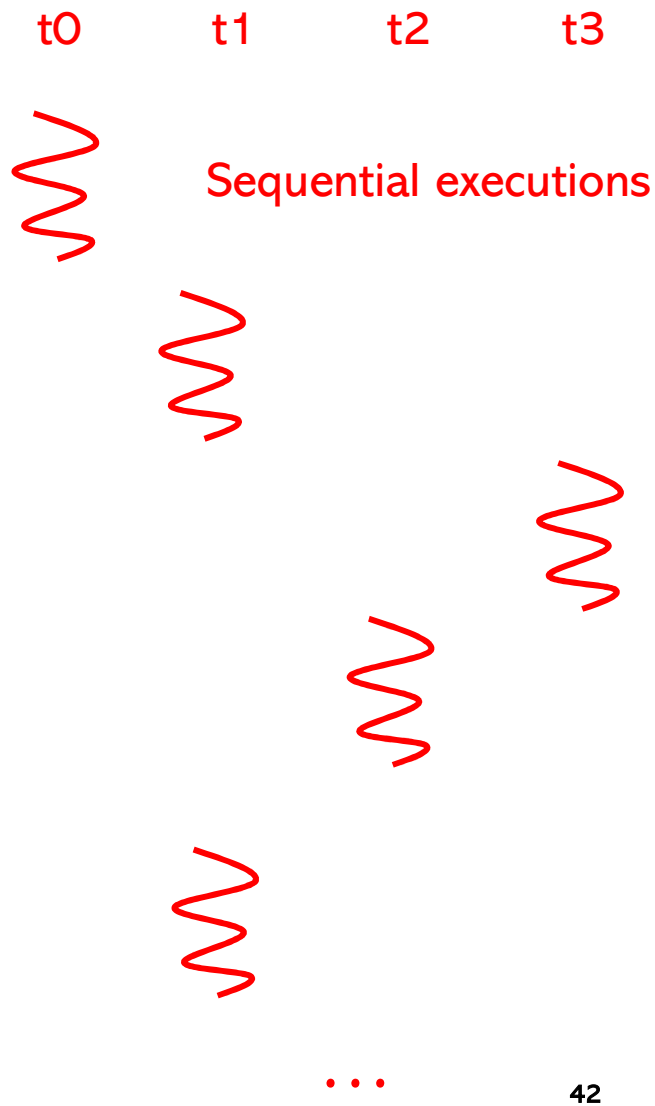
```
#include <mutex>
std::mutex global_mutex;

void inc(int* output)
{
  global_mutex.lock();
  (*output)++; //critical section
  global_mutex.unlock();
}

void worker(int* input, int start, int size, int*
output)
{
  for(int i=0;i<size;i++) {
    if(input[start+i]==0) {
      inc(output);
    }
  }
}
```

```
there are 1048576 zeros
```

Mutex Serializes Computations

```
#include <mutex>
std::mutex global_mutex;

void inc(int* output)
{
  global_mutex.lock();
  (*output)++; //protected by the global mutex
  global_mutex.unlock();
}

void worker(int* input, int start, int size, int*
output)
{
  for(int i=0;i<size;i++) {
    if(input[start+i]==0) {
      inc(output);
    }
  }
}
```

```
there are 1048576 zeros
```



A Better Solution (1)

```
void worker2(int* input, int start, int size, int* output)
{
        int localCount=0;
        for(int i=0;i<size;i++) {
                if(input[start+i]==0) {
                        localCount++;
                }
        }
        global_mutex.lock();
        (*output) += localCount;
        global_mutex.unlock();
}
```

only one critical section per thread

- Reduce the amount of work done in critical sections

```
there are 1048576 zeros
```

A Better Solution (2)

```
#include <iostream>
#include <thread>
#include <vector>

void worker3(int* input, int start, int
size, int* output)
{
        (*output)=0;
        for(int i=0;i<size;i++) {
                if(input[start+i]==0) {
                        (*output)++;
                }
        }
}
```

same code

```
int main()
{
  const int N = 1048576;
  const int NT = 4;
  int *array= new int[N];

  for(int i=0;i<N;i++) {
    array[i] = 0;
  }

  int count=0;
  std::vector<std::thread> threads;
  int localCount[NT];
    for(int t=0;t<NT;t++) {
    int size=N/NT;
    int start = t*size;
    threads.push_back(std::thread(worker3,
      array, start, size, &(localCount[t])));
        }
        for(std::thread& thread:threads) {
                thread.join();
        }

        for(int t=0;t<NT;t++) {
                count+=localCount[t];
        }
  std::cout<<"there are "<<count<<"
zeros"<<std::endl;
}
```

provide local variable as pointers

main thread adds them together into “count”

- Not parallelizing certain computation can be sometimes better
- When there are too many threads, performing a tree-reduction can be helpful (page14)

```
there are 1048576 zeros
```

Be careful on using mutex

```
std::mutex global_mutex;

void inc(int* output)
{
  global_mutex.lock();
  (*output)++; //critical section
  //global_mutex.unlock();  //FORGOT TO UNLOCK!
}

void worker(int* input, int start, int size, int*
output)
{
  for(int i=0;i<size;i++) {
    if(input[start+i]==0) {
      inc(output);
    }
  }
}
```

- Carelessly using lock & unlock results in a deadlock situation
  - No one can proceed

A safe alternative way on using mutex

```
std::mutex global_mutex;

void inc(int* output)
{
  std::lock_guard<std::mutex> guard(global_mutex);
  (*output)++;
}

void worker(int* input, int start, int size, int*
output)
{
  for(int i=0;i<size;i++) {
    if(input[start+i]==0) {
      inc(output);
    }
  }
}
```

- lock_guard
  - Locks on construction
  - Unlocks on destruction
    - When is the above guard destroyed?
- RAII scheme
  - Resource Allocation Is Initialization
  - A relatively modern scheme

Deadlock

```
class int_wrapper
{
  public:
    int_wrapper(int val):val(val){}
    std::mutex m;
    int val;
};

void swap(int_wrapper& v1,
              int_wrapper& v2)
{
  v1.m.lock();
  v2.m.lock();
  int tmp = v1.val;
  v1.val = v2.val;
  v2.val = tmp;
  v1.m.unlock();
  v2.m.unlock();
}
```

```
int main()
{
  int_wrapper a(0);
  int_wrapper x(1);

  for(int i=0;i<10000;i++) {
    std::cout<<"start iteration "<<i<<std::endl;
    std::thread t1(swap, std::ref(x), std::ref(a)) ;
    std::thread t2(swap, std::ref(a), std::ref(x)) ;

    t1.join();
    t2.join();
    std::cout<<"done a: "<<a.val
             <<", x: "<<x.val<<std::endl;
  }
    return 0;
}
```

- Results in a deadlock
- NOTE: std::ref is needed because std::thread constructor takes parameters as value
  - Even though the function swap has reference parameters

Deadlock – solution

- Simple solution:
  - If you have to lock multiple mutexes, always lock them in the same order!
  - But which order???
    - You can make up a rule, but sharing the rule with other people will be difficult

Avoiding Deadlock

```
void swap(int_wrapper& v1, int_wrapper& v2)
{
  std::lock(v1.m, v2.m);
  int tmp = v1.val;
  v1.val = v2.val;
  v2.val = tmp;
  v1.m.unlock();
  v2.m.unlock();
}
```

- std::lock() can lock multiple mutexes, without deadlock

Avoiding Deadlock

```
class int_wrapper
{
  public:
    int_wrapper(int val):val(val){}
    std::mutex m;
    int val;
};

void swap(int_wrapper& v1,
              int_wrapper& v2)
{
  std::lock(v1.m, v2.m);
  int tmp = v1.val;
  v1.val = v2.val;
  v2.val = tmp;
  v1.m.unlock();
  v2.m.unlock();
}
```

```
int main()
{
  int_wrapper a(0);
  int_wrapper x(1);

  for(int i=0;i<10000;i++) {
    std::cout<<"start iteration "<<i<<std::endl;
    std::thread t1(swap, std::ref(x), std::ref(a)) ;
    std::thread t2(swap, std::ref(a), std::ref(x)) ;

    t1.join();
    t2.join();
    std::cout<<"done. a: "<<a.val
             <<", x: "<<x.val<<std::endl;
  }
    return 0;
}
```

- std::lock() can lock multiple mutexes, without deadlock
  - It has its own ordering rule inside, so not conflicts (often in the pointer address order)

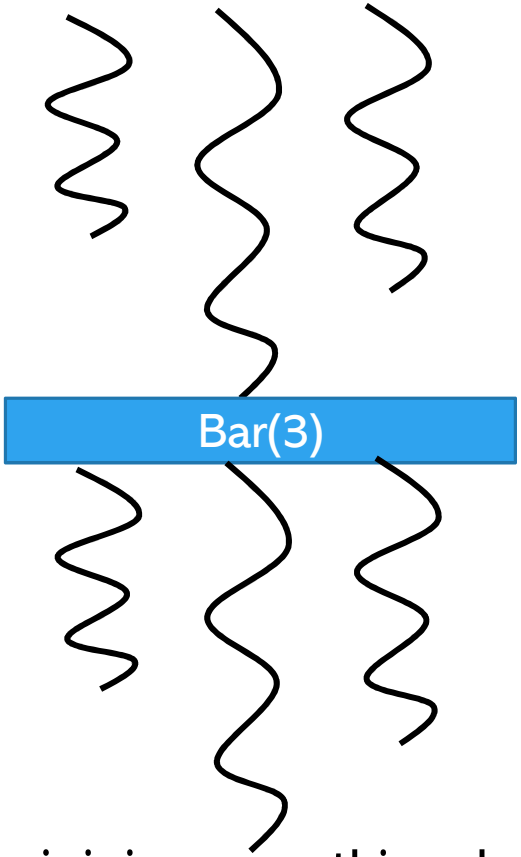
Atomics

```
#include <iostream>
#include <thread>
#include <vector>
#include <atomic>
#include <assert.h>

std::atomic<int> output

void worker(int* input, int start, int size)
{
  for(int i=0;i<size;i++) {
    if(input[start+i]==0) {
      output+=1;
    }
  }
}
```

- If the job between lock-unlock is very simple, you can use std::atomic
- There are special hardware instructions for this
- And it's much faster!
  - Warning: still *much* slower than normal operations
- Use atomic on short operations, use locks when the critical section is long

Barrier

- Barrier is a form of global synchronization



Needs 3 threads to push it away

- Similar to joining everything, but without terminating the threads
- Started being supported since C++20
  - With lower version compilers, use boost library

Using barrier

```
…
#include <barrier>
const int N=1000*16;
const int NT = 4;
std::barrier bar(NT);

void worker(int* input, int start, int size, int* output)
{
  for(int i=0;i<size;i++) {
    if(input[start+i]==0) {
      (*output)++;
    }
    if(i%1000==0) {
      bar.arrive_and_wait();
      std::cout<<"thread starting at "<<start<<" passed bar at i="<<i<<std::endl;
    }
  }
}
int main()
{
  int *array = new int[N];
…
  int count=0;
  std::vector<std::thread> threads;
  for(int t=0;t<NT;t++) {
    int size = N/NT;
    int start = size*t;
    threads.push_back(std::thread(worker, array, start, size, &count));
  }
…
```

- Compile with -std=c++20

Thread-safety

- If your code manipulates some shared data,
  and yet it is okay to execute the code with multiple threads
- Then your code is thread-safe
- Most of the data structures are not thread-safe
- You can simply make it thread-safe
  - Add a lock to it

Queue

```
template<typename T>
class queue
{
  std::queue<T> _queue;
  std::mutex mtx;
  public:

  queue()
  {}

  void push(T value)
  {

    _queue.push(value);
  }

  T pop()
  {

    T res = _queue.front();
    _queue.pop();
    return res;
  }
};
```

- Works, but not thread-safe

Thread-safe Queue

```
template<typename T>
class thread_safe_queue
{
  std::queue<T> _queue;
  std::mutex mtx;
  public:

  thread_safe_queue()
  {}

  void push(T value)
  {
    std::lock_guard<std::mutex> lock(mtx);
    _queue.push(value);
  }

  T pop()
  {
    std::lock_guard<std::mutex> lock(mtx);
    T res = _queue.front();
    _queue.pop();
    return res;
  }
};
```

- Safe, but inefficient

Linked-list Queue

```
template<typename T>
class linked_list_queue
{
  class node
  {
    public:
    node(T init_value):value(init_value)
    {}
    T value;
    node* next;
  };

  node* head;
  node* tail;
  public:

  linked_list_queue()
  {
    head = tail = new node(0); //dummy node
  }
```

```
  void push(T value)
  {
    node* tmp = new node(value);
    tmp->next = nullptr;
    tail->next = tmp;
    tail = tmp;
  }

  bool pop(T& value)
  {
    node* old_head = head;
    node* new_head = old_head->next;
    if(new_head == nullptr) {
      return false;
    }
    value = new_head->value;
    head = new_head;
    delete old_head;

    return true;
  }
};
```

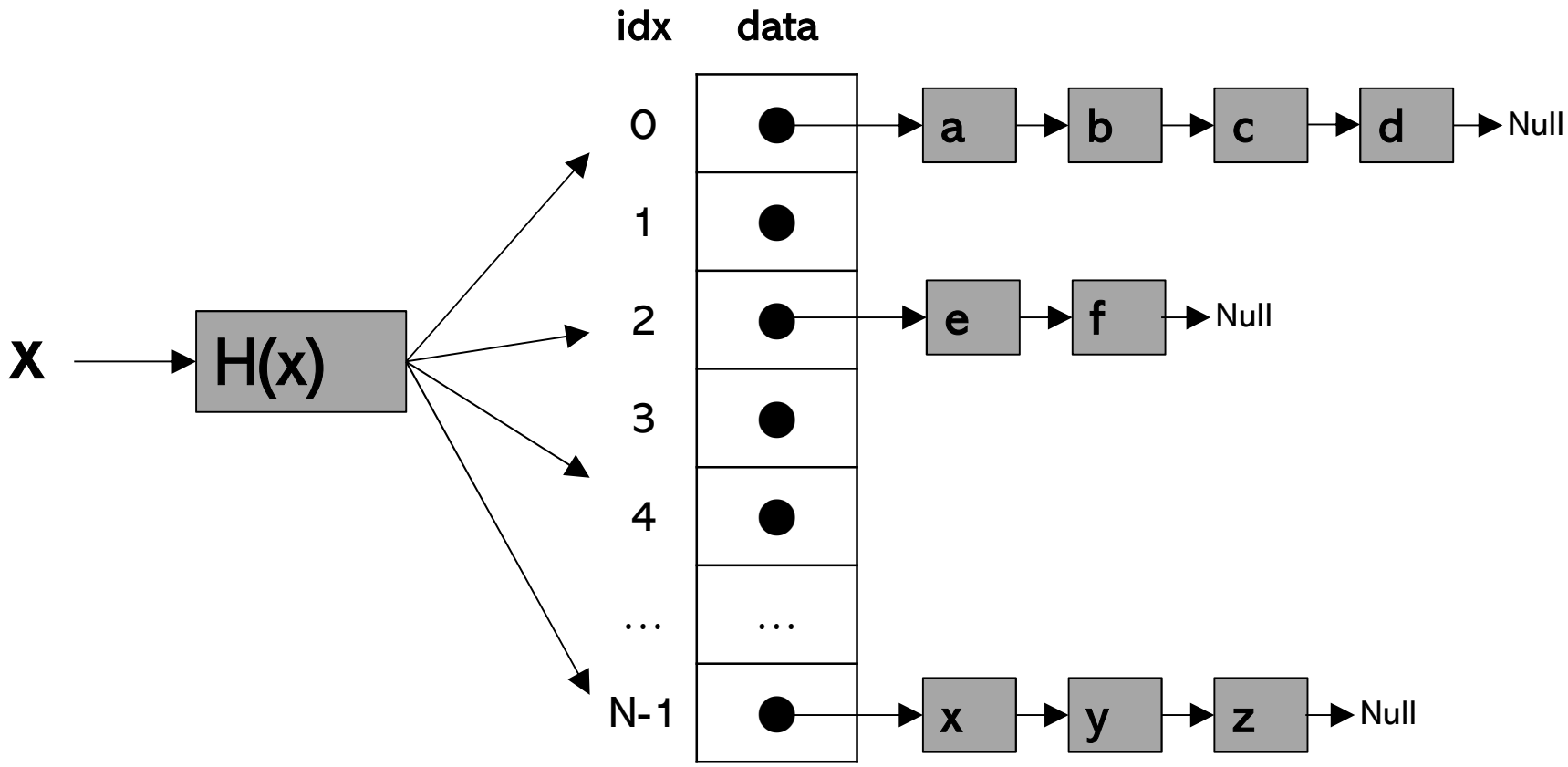
- Do we need to lock everything?
- push() only uses the tail
- pop() only used the head

Thread-safe Linked-list Queue

```
template<typename T>
class safe_llq
{
  class node
  {
    public:
    node(T init_value):value(init_value)
    {}
    T value;
    node* next;
  };

  node* head;
  node* tail;
  std::mutex head_mtx;
  std::mutex tail_mtx;
  public:

  safe_llq()
  {
    head = tail = new node(0); //dummy node
  }

  void push(T value)
  {
    node* tmp = new node(value);
    tmp->value = value;
    tmp->next = nullptr;
    std::lock_guard<std::mutex> tail_lock(tail_mtx);
    tail->next = tmp;
    tail = tmp;
  }

  bool pop(T& value)
  {
    std::lock_guard<std::mutex> head_lock(head_mtx);
    node* old_head = head;
    node* new_head = old_head->next;
    if(new_head == nullptr) {
      return false;
    }
    value = new_head->value;
    head = new_head;
    delete old_head;

    return true;
  }
};
```

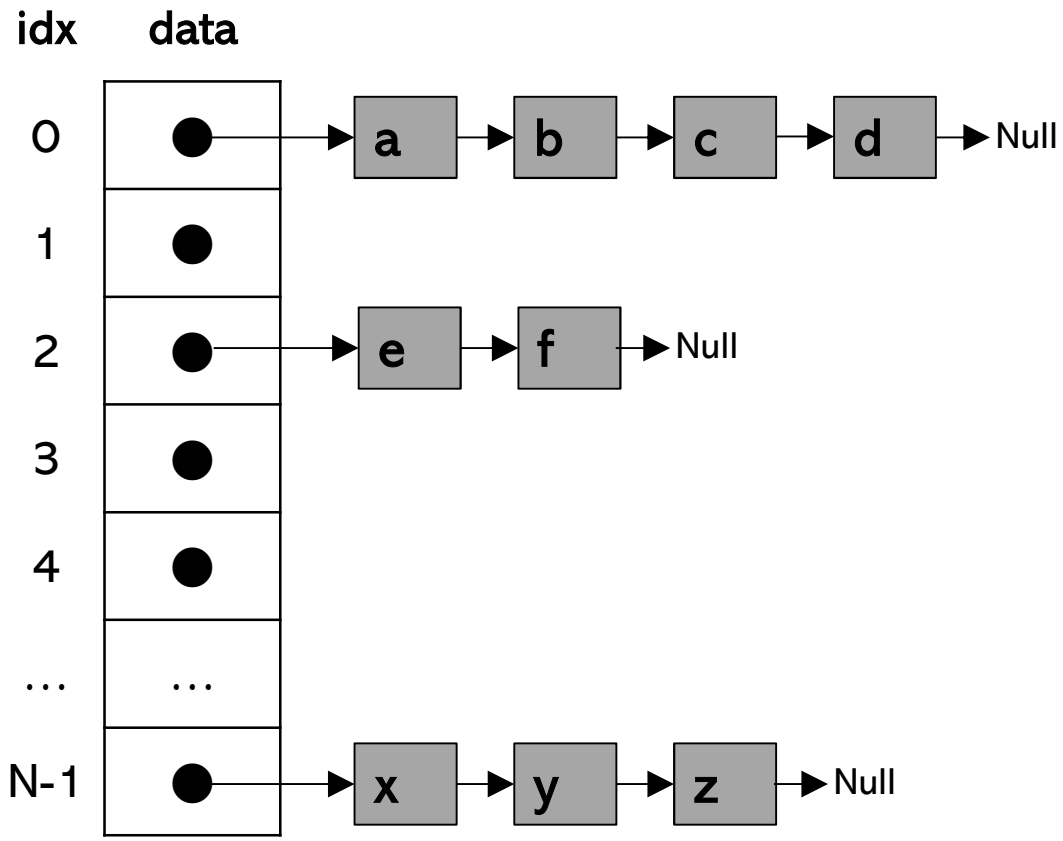
- Now this means that push() and pop() can happen at the same time
- This is called fine-grained locking

Thread-safe Hash Table



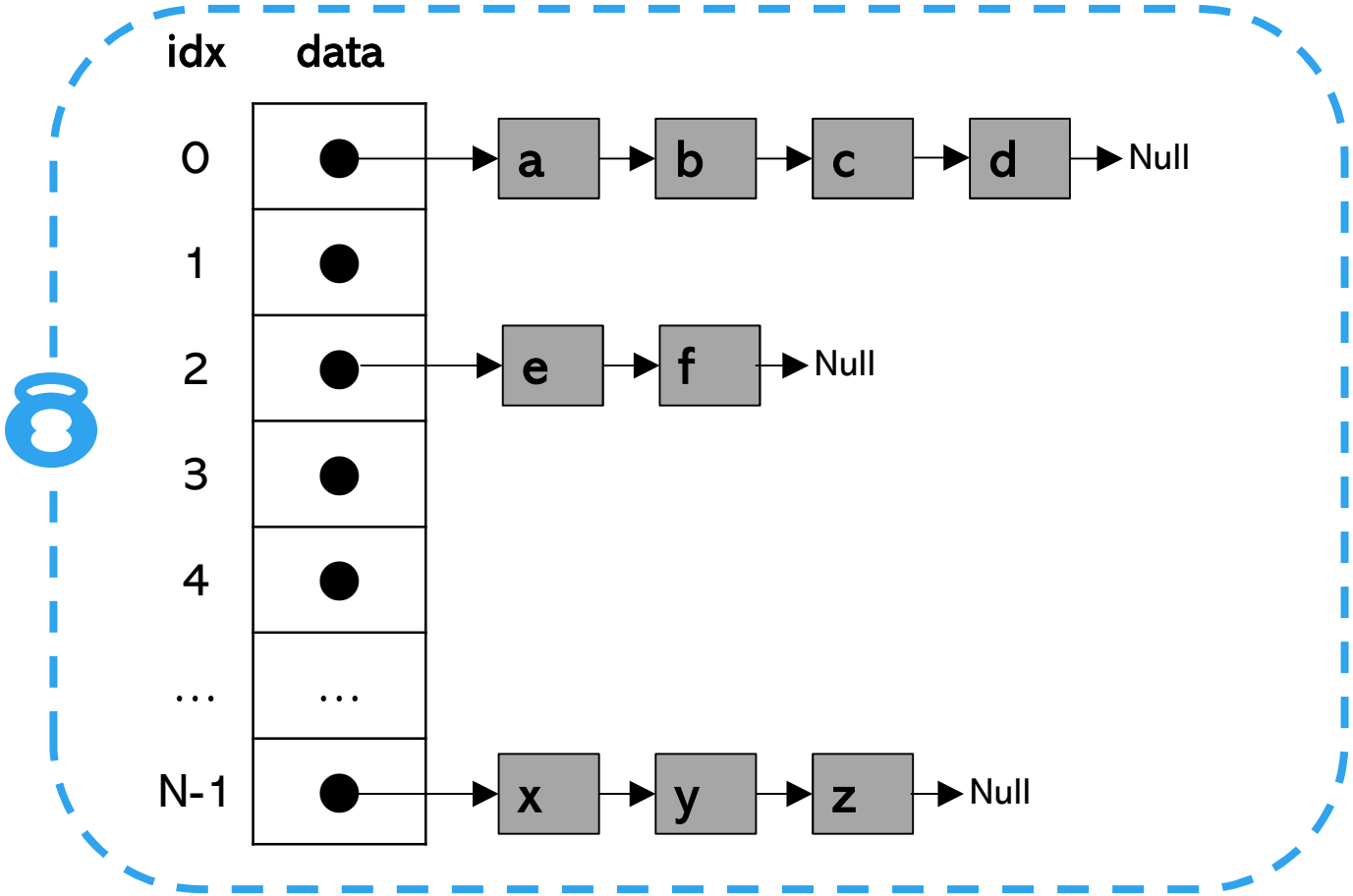
- Have you used hashtables?
  - You should have!
  - It's often practically the fastest data structure
    - O(1) write, O(1) read

Thread-safe Hash Table



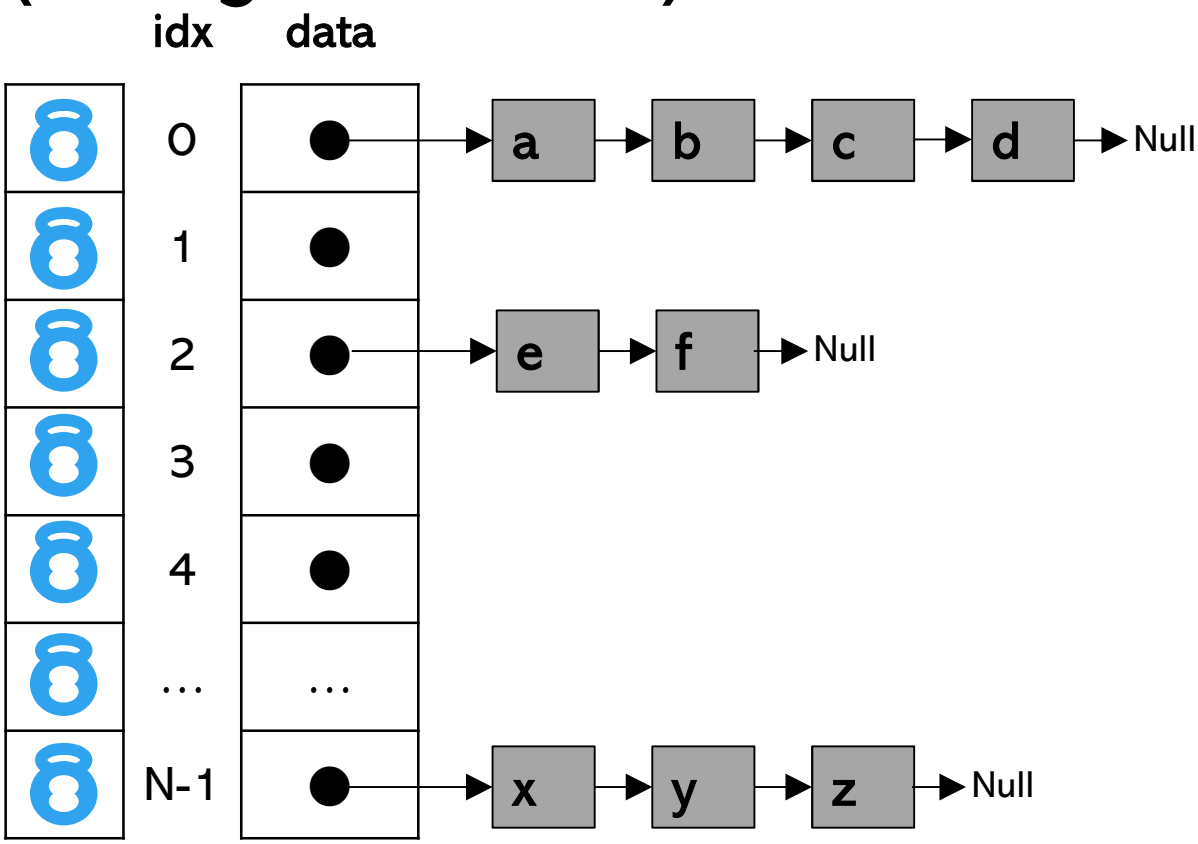
- Q: How do you make a hashtable thread-safe?

Thread-safe Hash Table (global lock)



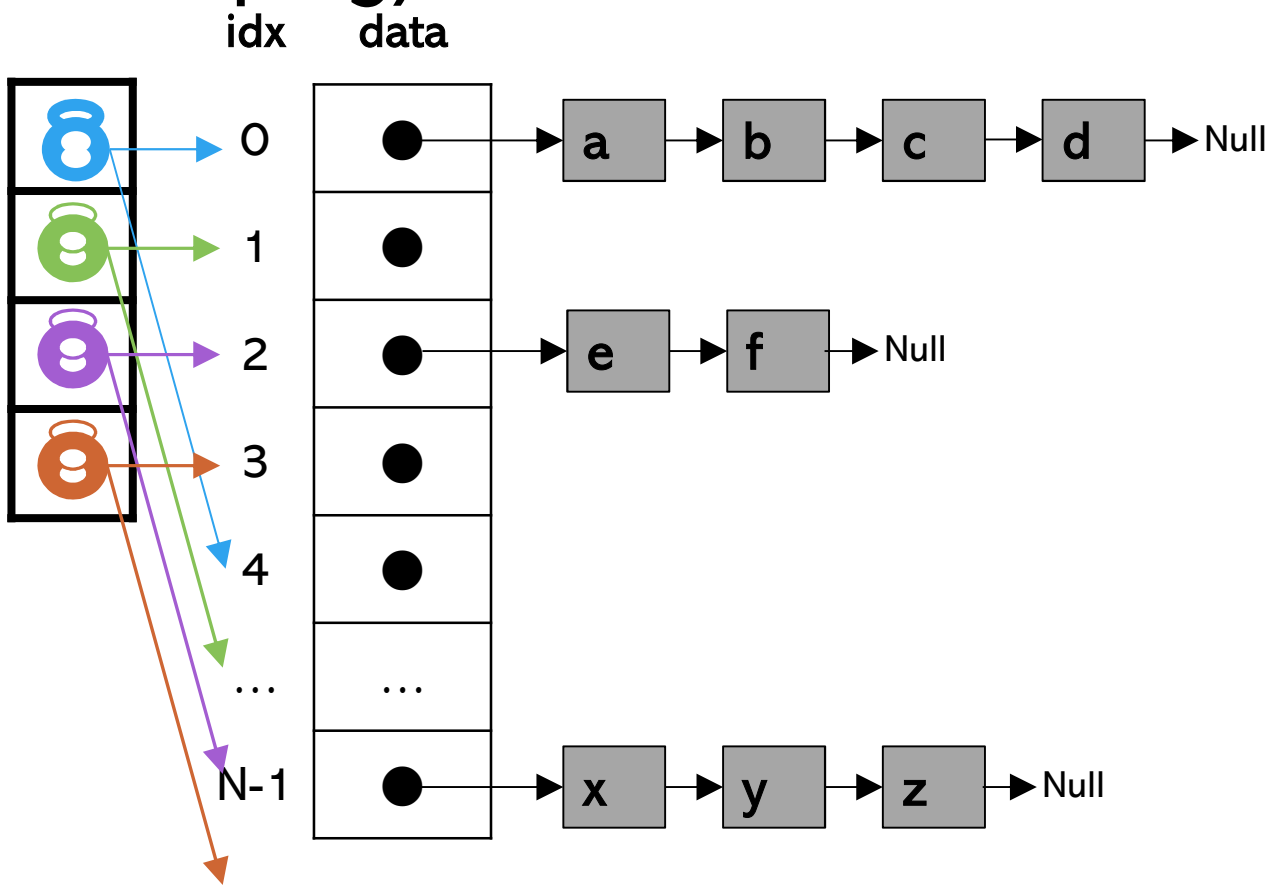
- Q: How do you make a hashtable thread-safe?
- A1: Easy! Put a lock!
  - Slow, obviously

Thread-safe Hash Table (Fine-grained lock)



- Q: How do you make a hashtable thread-safe?
- A2: Fine-grained locks!!
  - Fast, but too much space overhead! O(B) : B is number of buckets
  - Sometimes it's okay, sometimes it's not

Thread-safe Hash Table (lock striping)



- A3: Lock striping
  - Use array of locks (size L)
  - Lock l manages item i with $i \% L = l$
  - Trade-off between performance and space

Summary

- Use threads for multi-cores
- Locks, atomics and barriers to avoid race condition
  - But they are slow !
- The best way is to change your algorithm such that least amount (maybe none) of locks, atomics and barriers are used

Self-study Homework

1. How do you implement a lock?
2. Implement parallel reduction in various ways and measure speedup with various number of threads
3. Implement thread-safe hashtables and compare performance